

Understanding Cohesion and Coupling

Software Design Principles

What is Cohesion?

- Cohesion represents the relationship within a module, often referred to as intra-module relationship.
- Increased cohesion is considered good for software design.
- It represents the functional strength of modules.
- A highly cohesive module provides the best software quality.
- Strong cohesion is not aesthetic minimalism; it is structural intelligence.
- Cohesion reduces cognitive load, improves testability, and supports evolutionary architecture.

Types of Cohesion

There are six primary types of cohesion:

- Temporal Cohesion (Lowest)
- Procedural Cohesion
- Communication Cohesion
- Sequential Cohesion
- Layer Cohesion
- Functional Cohesion (Highest)

Remember:

From a design philosophy perspective, cohesion increases as the unifying principle becomes more conceptual and purpose-driven rather than incidental (time, order, or data coincidence).

Details on Cohesion Types

- **Functional / Logical:** A subsystem performing logically related tasks.
- **Temporal:** Tasks that must be executed within a specific time span.
- **Procedural:** Processing elements are related and must follow a specific order.
- **Communication:** Processing elements share the same data.
- **Sequential:** The output of one subsystem serves as the input for another.

Functional Cohesion in Software Design

Examples:

- A function dedicated solely to computing payroll salary
- A module responsible only for user authentication
- A class that validates input data and performs no other task

Implications:

- Highest and most desirable form of cohesion
- Enhances maintainability and testability
- Promotes clear responsibility and modular clarity

Procedural Cohesion in Software Design

Examples:

- A module executing tasks in a specific sequence (initialize → process → terminate)
- A script performing ordered system setup steps
- A function coordinating multiple unrelated steps in order

Implications:

- Order-based relationship rather than functional unity
- Moderate structural clarity
- Less flexible to change compared to functional cohesion

Temporal Cohesion in Software Design

Examples:

- Initialization routines executed during system startup
- Cleanup operations performed during shutdown
- Logging setup tasks grouped together at runtime entry

Implications:

- Activities grouped by timing rather than purpose
- Lower cohesion than procedural or functional types
- Modifications may introduce hidden side effects

Sequential Cohesion in Software Design

Examples:

- Output of data parsing used as input for validation
- A pipeline performing read $\rightarrow$ process $\rightarrow$ write operations
- Compiler phases such as lexical analysis followed by syntax analysis

Implications:

- Moderate to strong cohesion
- Logical data flow improves readability
- Changes in one stage may affect subsequent stages

Layer Cohesion in Software Design

Examples:

- Presentation layer handling only UI rendering logic
- Business layer implementing domain rules exclusively
- Data access layer managing database operations only

Implications:

- Promotes architectural separation of concerns
- Enhances scalability and system organization
- Improves testability through clear boundary definition

Communication Cohesion in Software Design

Examples:

- Multiple functions operating on the same customer record
- A module performing read and update operations on a shared dataset
- Reporting functions accessing the same transaction database

Implications:

- Groups operations around shared data
- Moderate maintainability
- Data structure changes impact all related functions

What is Coupling?

- Coupling represents the relationships between different modules, known as inter-module relationship.
- Increased coupling should be avoided during software design.
- It represents the level of independence among modules.
- Loosely coupled modules result in the best software.

Types of Coupling

There are six primary types of coupling:

- Data Coupling (Lowest)
- Stamp Coupling
- Control Coupling
- External Coupling
- Common Coupling
- Content Coupling (Highest)

Details on Coupling Types (Lowest to Highest)

- **Data:** Interaction via parameter passing or data interaction.
- **Stamp:** Modules share a composite data structure but use only specific parts.
- **Control:** Modules share related control data.
- **External:** External coupling is the dependency of a software module on externally defined systems, data formats, communication protocols, hardware constraints, operating environments, or third party systems that lie outside its architectural boundary.
- **Common:** Sharing of common or global data among modules.
- **Content:** One module uses data or control info maintained in another.

Data Coupling in Software Design

Examples:

- A function passing only required parameters (e.g., `calculateTotal(price, tax)`)
- A module sending primitive data types to another module
- API endpoints exchanging only necessary JSON fields

Implications:

- Lowest and most desirable form of coupling
- Improves modularity and maintainability
- Changes in one module minimally affect others

Stamp Coupling in Software Design

Examples:

- Passing an entire object when only one attribute is required
- Sending a full record structure to a function using partial data
- Sharing database entity objects across multiple layers

Implications:

- Hidden dependencies on unused data fields
- Structural changes ripple across modules
- Reduced encapsulation and flexibility

Control Coupling in Software Design

Examples:

- A module passing a flag to dictate another module's behavior
- A function receiving a mode parameter to switch logic paths
- UI layer sending control codes to backend services

Implications:

- Reduces module independence
- Increases complexity of understanding behavior
- Modifications in control logic propagate across modules

External Coupling in Software Design

Examples:

- Dependency on third-party APIs (e.g., payment gateways, cloud services)
- Fixed file formats mandated by regulatory authorities
- OS-specific configurations or hardware dependencies

Implications:

- Reduced system stability due to lack of control over external changes
- Increased integration and maintenance risk
- Higher testing complexity

Common Coupling in Software Design

Examples:

- Multiple modules accessing global variables
- Shared configuration files modified by different components
- Static shared memory accessed across modules

Implications:

- High risk of unintended side effects
- Difficult debugging due to shared state mutations
- Poor scalability and concurrency challenges

Content Coupling in Software Design

Examples:

- One module directly modifying another module's internal data
- Branching into the middle of another module's logic
- Accessing private variables or internal structures

Implications:

- Highest and most undesirable form of coupling
- Severe violation of encapsulation principles
- Extremely fragile and difficult to maintain systems

Cohesion and Coupling Code Structures

Functional Cohesion

Every method contributes to one well-defined responsibility.

```
class AuthenticationModule {  
public:  
    bool validateCredentials(const std::string& user,  
                             const std::string& pass);  
    std::string generateToken(const std::string& userId);  
    void logAuthenticationEvent(const std::string& userId);  
};
```

All methods serve one purpose: authentication.

Sequential Cohesion

Output of one function becomes input to the next.

```
class DataProcessingModule {  
public:  
    std::string readRawData();  
    std::string cleanData(const std::string& raw);  
    std::string transformData(const std::string& cleaned);  
};
```

Pipeline structure: read → clean → transform.

Communicational Cohesion

Operations grouped because they operate on the same data.

```
class CustomerRecordModule {  
private:  
    Customer customer;  
  
public:  
    void updateAddress(const std::string& addr);  
    void updatePhone(const std::string& phone);  
    void printCustomerSummary();  
};
```

All methods revolve around the same data structure.

Procedural Cohesion

Grouped because of control sequence, not functional unity.

```
class SystemStartupModule {  
public:  
    void initializeMemory();  
    void loadConfiguration();  
    void displayWelcomeScreen();  
};
```

Steps follow an order but are conceptually unrelated.

Temporal Cohesion

Grouped because executed at the same time.

```
class ShutdownModule {  
public:  
    void closeDatabaseConnection();  
    void saveLogs();  
    void releaseMemory();  
};
```

All occur at shutdown — purpose differs.

Layer Cohesion

Module belongs to one architectural layer.

```
class UserRepository { // Data Layer
public:
    void saveUser(const User& user);
    User fetchUserById(int id);
};
```

All responsibilities belong strictly to data access.

Data Coupling

Modules interact via simple parameter passing.

```
class BillingModule {  
public:  
    double calculateTotal(double price, double tax);  
};
```

Only necessary data is passed.

Stamp Coupling

Entire object passed when only part is needed.

```
class OrderProcessor {  
public:  
    void processOrder(const Order& order);  
    // Uses only part of Order internally  
};
```

Passing full structure creates unnecessary dependency.

Control Coupling

One module controls another via flags.

```
class ReportGenerator {  
public:  
    void generateReport(bool detailedMode);  
};
```

Flag dictates internal logic branch.

Common Coupling

Shared global state.

```
// Global variable
extern int systemStatus;

class ModuleA {
public:
    void updateStatus();
};

class ModuleB {
public:
    void readStatus();
};
```

Multiple modules depend on shared global data.

Content Coupling

One module directly modifies internals of another.

```
class DatabaseModule {
public:
    int internalConnectionState;  // Should be private
};

class AdminModule {
public:
    void forceReset(DatabaseModule& db) {
        db.internalConnectionState = 0;
    }
};
```

Encapsulation is violated.

- Cohesion measures internal unity.
- Coupling measures external dependency.

Remember:

High cohesion + low coupling = Good architectural design.

- Architecture is visible in structure long before runtime behavior emerges.
- Software design is not about writing code that works. It is about shaping dependency so that the future remains affordable.